Explainable Breast Cancer Detection using Ensemble Learning

Individual Contribution Report

EPICS Phase – II

Submitted by

Suryanshi Ranawat
Register Number: 23BCE11634

in partial fulfillment of the requirements for the degree of

Bachelor of Technology
Computer Science and Engineering

VIT Bhopal University
Kothrikalan, Sehore
Madhya Pradesh

March 2026

Abstract

Breast cancer affects millions of women globally, and catching it early genuinely saves lives. Our team built a machine learning system that tries to help with detection while also being transparent about how it makes decisions – because in medicine, a black box prediction isn't helpful.

We experimented with several classification algorithms – Logistic Regression, Decision Trees, SVM, and Random Forest – and combined them using soft voting. The idea was that multiple models working together would be more reliable than any single one.

One thing we really focused on was making the system explainable. It doesn't just say "malignant" or "benign" – it provides confidence scores and shows which features influenced the decision. Our best models achieved 98.25% accuracy with ROC-AUC above 0.99.

Keywords: Breast Cancer Detection, Machine Learning, Ensemble Learning, Explainable AI, Data Preprocessing, Exploratory Data Analysis

Contents

1 MY CONTRIBUTION **1**
- 1.1 Overview . . . 1
- 1.2 Data Preprocessing . . . 1
  - 1.2.1 Handling Missing Values . . . 1
  - 1.2.2 Data Normalization and Scaling . . . 1
  - 1.2.3 Train-Test Split . . . 1
- 1.3 Exploratory Data Analysis . . . 2
  - 1.3.1 Statistical Analysis . . . 2
  - 1.3.2 Correlation Analysis . . . 2
  - 1.3.3 Feature Distribution Visualization . . . 2
  - 1.3.4 Key Features Identification . . . 3
- 1.4 Visualization Development . . . 3
  - 1.4.1 Correlation Heatmap . . . 3
  - 1.4.2 Feature Distribution Plots . . . 3
  - 1.4.3 Class Distribution Analysis . . . 3
- 1.5 Data Quality Assurance . . . 3

2 PROJECT OVERVIEW **4**
- 2.1 Problem Statement . . . 4
- 2.2 System Architecture . . . 4
- 2.3 Results Achieved . . . 4

3 TEAM CONTRIBUTIONS **5**
- 3.1 Team Members and Their Roles . . . 5
- 3.2 Collaboration . . . 5

4 LEARNING OUTCOMES **6**

5 CONCLUSION **7**

6 REFERENCES **8**

1. MY CONTRIBUTION

1.1 Overview

My main job in this project was **data preprocessing and exploratory data analysis (EDA)**. Basically, I took the raw dataset that Bhavya found and got it ready for the ML models. This might sound straightforward, but good preprocessing can make or break a project – garbage in, garbage out, as they say.

1.2 Data Preprocessing

Preprocessing is one of those things that doesn't sound glamorous but is absolutely critical. Here's what I focused on:

1.2.1 Handling Missing Values

First thing I did was check for missing data. You'd be surprised how many datasets have gaps:

- Went through all 30 features systematically
- Checked every single row (569 of them)
- Good news: WDBC is squeaky clean – not a single missing value
- Documented this so the team knew we didn't need imputation

1.2.2 Data Normalization and Scaling

This was probably the most important preprocessing step. I used standard scaling (Z-score normalization):

$$z_i = \frac{x_i - \mu}{\sigma} \tag{1}$$

where μ is the mean and σ is the standard deviation.
Here's what I actually did:

- Applied StandardScaler from scikit-learn to all features
- Made sure to fit the scaler *only* on training data (this prevents data leakage!)
- Applied the same transformation to test data
- Verified the scaled features had mean 0 and std 1

1.2.3 Train-Test Split

I set up the train-test split with stratification:

- Used an 80/20 split (455 training, 114 test samples)
- Stratified by diagnosis to keep class ratios the same in both sets

- Set a random seed so results are reproducible

Stratification ensures:

$$\frac{|y_{train}^{(c)}|}{|\mathcal{D}_{train}|} = \frac{|y_{test}^{(c)}|}{|\mathcal{D}_{test}|} = \frac{|y^{(c)}|}{|\mathcal{D}|} \tag{2}$$

1.3 Exploratory Data Analysis

EDA was actually the fun part – getting to know the dataset before building models.

1.3.1 Statistical Analysis

I ran a bunch of statistics on all the features:

- Calculated basics: mean, median, std, min, max, quartiles
- Looked at how spread out the values were for each feature
- Flagged potential outliers using IQR (some were pretty extreme)
- Compared statistics between benign and malignant groups – this was really revealing

1.3.2 Correlation Analysis

I built correlation matrices to understand how features related to each other:

- Made heatmaps for all 30 features (pretty busy-looking, honestly)
- Found that radius, perimeter, and area are super correlated (makes sense mathematically)
- Noted multicollinearity issues that might affect linear models
- Narrowed down to top 15 features for a cleaner visualization

The high correlations between geometric features weren't surprising – area is basically radius squared, after all.

1.3.3 Feature Distribution Visualization

I created visualizations to see how benign and malignant cases differ:

- Histograms comparing the two classes – some features showed really clear separation
- Box plots to see spread and outliers
- Violin plots for more detailed distribution shapes
- Identified which features would likely be most predictive

1.3.4 Key Features Identification

From all this analysis, the most discriminating features seemed to be:

1. **Worst Area**: Huge difference between benign and malignant
2. **Worst Concave Points**: Malignant tumors have more of these
3. **Worst Radius**: Malignant tumors tend to be bigger
4. **Mean Concave Points**: Consistently different between classes
5. **Worst Perimeter**: Related to tumor size

1.4 Visualization Development

I made several plots that ended up in our reports:

1.4.1 Correlation Heatmap

The heatmap showing correlations among the top 15 features was pretty informative. You could really see clusters of related features.

1.4.2 Feature Distribution Plots

I created side-by-side plots comparing benign and malignant distributions. For features like "worst concave points," you can literally see the two classes barely overlap – that's a good sign for classification.

1.4.3 Class Distribution Analysis

I also visualized the class balance:

- Benign: 357 cases (62.7%)
- Malignant: 212 cases (37.3%)

It's imbalanced, but not severely. This informed our decision to use stratification and to track metrics like recall that account for class imbalance.

1.5 Data Quality Assurance

Before declaring the data "ready," I did some final checks:

- Confirmed all columns had correct dtypes (no strings hiding in numeric columns)
- Searched for duplicate rows (none found)
- Sanity-checked feature ranges against what medical literature says is reasonable
- Documented every preprocessing step so others could reproduce it

2. PROJECT OVERVIEW

2.1 Problem Statement

We wanted to build a system that could distinguish benign from malignant tumors, give confidence scores, and explain its reasoning. In medicine, just getting a "yes" or "no" isn't helpful – doctors need to understand the why.

2.2 System Architecture

We organized the project into four layers:

1. **Data Layer**: Loading and preprocessing (my domain)
2. **Model Layer**: The classification algorithms
3. **Ensemble Layer**: Soft voting combination
4. **Output Layer**: Metrics and visualizations

2.3 Results Achieved

We got some solid results:

- LR and SVM both achieved 98.25% accuracy
- ROC-AUC above 0.99 for the best models
- 98.61% recall – catching almost all the cancer cases
- Cross-validation confirmed the results were robust

I like to think my preprocessing work contributed to these numbers – clean, well-scaled data makes everything downstream work better.

3. TEAM CONTRIBUTIONS

3.1 Team Members and Their Roles

- **Bhavya Sharma**: Found the dataset and explained the medical terminology
- **Suryanshi Ranawat (Me)**: Data preprocessing and exploratory analysis
- **Ujjwal Mishra**: Implemented the ML algorithms
- **Medhansh Singh Kushwaha**: Built the ensemble system
- **Soumyadeep Chakravarti**: Coordinated everything and wrote the final report

3.2 Collaboration

Everyone contributed their part, and my work was kind of the bridge between data acquisition and model training. After Bhavya got the dataset ready, I cleaned and analyzed it, then passed it to Ujjwal and Medhansh for the ML work. We'd check in regularly to make sure my outputs matched what they needed as inputs.

4. LEARNING OUTCOMES

Here's what I learned from working on this project:

1. **Data Preprocessing**: I really understand normalization and train-test splitting now, including why data leakage is such a big deal
2. **Exploratory Data Analysis**: I got much better at using statistics and visualizations to actually understand data before modeling
3. **Python Libraries**: Got comfortable with pandas, numpy, matplotlib, seaborn, and scikit-learn's preprocessing modules
4. **Feature Analysis**: Learned to identify which features are actually useful through correlation and distribution analysis
5. **Data Quality**: Developed a systematic approach to checking data quality – not just trusting that it's clean
6. **Healthcare Data**: Gained appreciation for how careful you need to be with medical data – preprocessing choices can affect real diagnostic outcomes

5. CONCLUSION

My part of this project was all about getting the data ready – preprocessing and exploratory analysis. It doesn't sound as exciting as building ML models, but honestly, it's where a lot of the real work happens. Badly prepared data would have torpedoed everything downstream.

The EDA I did was genuinely useful. The visualizations showed which features had the clearest separation between classes, and understanding feature correlations helped avoid redundant information. The preprocessing pipeline I built followed best practices (fitting only on training data, stratified splits) to prevent the kinds of subtle bugs that can invalidate results.

I'm happy with how the project turned out – 98.25% accuracy is solid, and I think my careful data preparation contributed to that. It was also interesting to work on something with real medical relevance rather than just toy datasets.

This EPICS project taught me that data work is foundational. You can have the fanciest model architecture in the world, but if your data is messy or your preprocessing leaks information, none of it matters.

6. REFERENCES

References

[1] W. H. Wolberg, W. N. Street, and O. L. Mangasarian, "Breast Cancer Wisconsin (Diagnostic) Dataset," UCI Machine Learning Repository, 1995. [Online]. Available: https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic)

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://scikit-learn.org

[3] Pandas, "Python Data Analysis Library," 2023. [Online]. Available: https://pandas.pydata.org

[4] Matplotlib, "Visualization with Python," 2023. [Online]. Available: https://matplotlib.org

[5] S. Chakravarti et al., "EPICS Project: Explainable Breast Cancer Detection," GitHub, 2026. [Online]. Available: https://github.com/Soumyadeep-Chakravarti/EPICS_Project